University of Exeter

ECM2433: The C Family Inheritance

Reminder: classe access specifiers

```
class class-name
{
    private variables and functions
  access-specifier:
    variables and functions
  access-specifier:
    variables and functions
  access-specifier:
    variables and functions
};
```

- private (default)
  can only be accessed within the class
- public
  can be accessed by any part of the program
- protected
  accessed by inheritors

Inheritance: general form

General form for class inheritance:

```
class derived-class-name : access-specifier base-class-name
{ /* body of class */ };
```

For example:

```
class MySpecialString: public Stringy
{
    char type;
public:
    MySpecialString(char, const char *);
};
```

```
// Stringy.h
class Stringy
{
    std::string thestring;
public:
    Stringy(const char *);
    ~Stringy(); int
    len();
};
```

Types of inheritance

Type	public
Example	`class D: public Class B`
Public members	Are public in the derived class
Protected members	Protected in the derived class
Private members	Not accessible in derived class

Protected members in a class are similar to private members as they cannot be accessed from outside the class. But they can be accessed by derived classes while private members cannot.

Types of inheritance

Type	public	protected
Example	`class D: public Class B`	`class D: protected Class B`
Public members	Are public in the derived class	Protected in the derived class
Protected members	Protected in the derived class	Protected in the derived class
Private members	Not accessible in derived class	Not accessible in derived class

Protected members in a class are similar to private members as they cannot be accessed from outside the class. But they can be accessed by derived classes while private members cannot.

Types of inheritance

Type	public	protected	private
Example	`class D: public Class B`	`class D: protected Class B`	`class D: private Class B`
Public members	Are public in the derived class	Protected in the derived class	Private in the derived class
Protected members	Protected in the derived class	Protected in the derived class	Private in the derived class
Private members	Not accessible in derived class	Not accessible in derived class	Not accessible in derived class

Protected members in a class are similar to private members as they cannot be accessed from outside the class. But they can be accessed by derived classes while private members cannot.

Inheritance: public

```
class base
{
    int i;
protected:
    int j;
public:
    void set(int a, int b) { i=a; j=b;}
    void show() { printf("i=%d, j=%d\n",i,j); }
};

class derived: public base
{
    int k;
public:
    derived(int x) { k=x; i=0; j=0; }
    void showk()    { printf("k=%d\n",k); }
    void showAll() { show(); showk();}
};
```

error: i is private to base

```
derived obj(3);

obj.set(1,2);
obj.show();
obj.showk();
```

base
base
derived

Inheritance: protected

University of Exeter

```
class base
{
    int i;
protected:
    int j;
public:
    void set(int a, int b) { i=a; j=b;}
    void show() { printf("i=%d, j=%d\n",i,j); }
};

class derived: protected base
{
    int k;
public:
    derived(int x) { k=x; i=0; j=0; }
    void showk()   { printf("k=%d\n",k); }
    void showAll() { show(); showk();}
};
```

error: set() and show() are **protected** in derived

error: i is private to base

derived class can access show() and accessible to child of derived

```
derived obj(3);

obj.set(1,2);
obj.show();
obj.showk();
obj.showAll();
```

error

error

derived

Inheritance: private

```
class base
{
   int i;
protected:
   int j;
public:
   void set(int a, int b) { i=a; j=b;}
   void show() { printf("i=%d, j=%d\n",i,j); }
};

class derived: private base
{
   int k;
public:
   derived(int x) { k=x; i=0; j=0; }
   void showk()    { printf("k=%d\n",k); }
   void showAll()  { show(); showk();}
};
```

error: i is private to base

error: set() and show() are **private** in derived

derived class can access show() although private, but not accessible to child of derived

```
derived obj(3);

obj.set(1,2);
obj.show();
obj.showk();
obj.showAll();
```

error

error

derived

Multiple inheritance

```
class base1
{
 protected:
    int x;
 public:
    void showx() { printf("x=%d\n",x); }
};
class base2
{
 protected:
    int y;
 public:
    void showy() { printf("y=%d\n",y); }
};
class derived: public base1, public base2
{
 public:
    void set(int i, int j) { x=i; y=j; }
};
```

```
int main ()
{
    derived obj;

    obj.set(1,2);
    obj.showx();
    obj.showy();
}
```

derived
base1
base2

```
x=1
y=2
```

Inheritance access: good practice

- Use public inheritance unless there is a good reason to do otherwise

- Protected attributes can be accessed from derived classes
  - If you change the semantics of the attribute in the base class, then derived classes will probably have to change too
  - Therefore, prefer private members to insulate the class from changes by derived classes.
  - But may require a larger interface

Constructors and destructors

Two questions:

1. When are base and derived class constructors and destructors called?
2. How can parameters be passed to base class constructor functions?

Constructors and destructors

University of Exeter

simple inheritance

```
class base
{
 public:
    base()  {std::cout << "Constructing base" << std::endl; }
    ~base() {std::cout << "Destructing base" << std::endl; }
};

class derived: public base
{
 public:
   derived()  {std::cout << "Constructing derived" << std::endl; }
   ~derived() {std::cout << "Destructing derived" << std::endl; }
};

// This program does nothing but construct and destruct
int main ()
{
   derived obj;
   return 0;
}
```

Constructors and destructors

simple inheritance

```
class base
{
 public:
    base()  {std::cout << "Constructing base" << std::endl; }
    ~base() {std::cout << "Destructing base" << std::endl; }
};

class derived: public base
{
 public:
   derived()  {std::cout << "Constructing derived" << std::endl; }
   ~derived() {std::cout << "Destructing derived" << std::endl; }
};

// This program does nothing but construct and destruct
int main ()
{
   derived obj;
   return 0;
}
```

```
Constructing base
Constructing derived
Destructing derived
Destructing base
```

Constructors and destructors

General rules:

- Constructors are executed in order of derivation.
- Destructors are executed in reverse order of derivation.

Chain inheritance

University of Exeter

chain inheritance

```
class base
{
 public:
   base()   { std::cout << "Constructing base" << std::endl; }
   ~base()  { std::cout << "Destructing base" << std::endl; }
};

class derived1: public base
{
 public:
   derived1()   { std::cout << "Constructing derived1" << std::endl; }
   ~derived1()  { std::cout << "Destructing derived1" << std::endl; }
};

class derived2: public derived1
{
 public:
   derived2()   { std::cout << "Constructing derived2" << std::endl; }
}; ~derived2()  { std::cout << "Destructing derived2" << std::endl; }

int main () // This program does nothing but construct and destruct
{
   derived2 obj;
   return 0;
}
```

```
Constructing base
Constructing derived1
Constructing derived2
Destructing derived2
Destructing derived1
Destructing base
```

Multiple inheritance

University of Exeter

Multiple inheritance

```
class base1
{
 public:
   base1()   { std::cout << "Constructing base1" << std::endl; }
   ~base1()  { std::cout << "Destructing base1" << std::endl; }
};

class base2
{
 public:
   base2()       {   std::cout   << "Constructing base2" << std::endl; }
   ~base2()      {   std::cout   << "Destructing base2" << std::endl; }
};

class derived: public base1, public base2
{
 public:
   derived()   { std::cout << "Constructing derived" << std::endl; }
   ~derived()  { std::cout << "Destructing derived" << std::endl; }
};

int main ()  // This program does nothing but construct and destruct
{
   derived obj; return 0;
}
```

```
Constructing base1
Constructing base2
Constructing derived
Destructing derived
Destructing base2
Destructing base1
```

Constructor parameters

```
class base
{
 protected: int i;
 public:
   base(int i) { this->i = i; std::cout << "Constructing base i = ” << i << std::endl }
   ~base()     { std::cout << "Destructing base" << std::endl; }

};
class derived: public base
{
   int j;
 public:
   derived(int x, int y);
   ~derived()  { std::cout << "Destructing derived" << std::endl; }
   void show() { printf("i=%d, j=%d\n", i, j); }
};

int main ()
{
   derived obj(3,4);
   obj.show(); return
   0;
}
```

```
Constructing base i = 4
Constructing derived
i=4, j=3
Destructing derived
Destructing base
```

```
derived::derived(int x, int y): base(y)
{
   j=x;
   cout << "Constructing derived" << endl;
}
```

Constructor parameters

University of Exeter

Multiple inheritance

```
class base1
{
 protected:
   int i;
 public:
   base1(int x)   { i=x; std::cout << "Constructing base1" << std::endl; }
   ~base1()       { std::cout << "Destructing base1" << std::endl; }
};

class base2
{
 protected:
   int j;
 public:
   base2(int y)   { j=y; std::cout << "Constructing base2" << std::endl; }
   ~base2()       { std::cout << "Destructing base2" << std::endl; }
};

class derived: public base1, public base2
{
   int k;
 public:
   derived(int x, int y, int z);
    ~derived()    { std::cout << "Destructing derived" << std::endl; }
    void show()   { printf("i=%d, j=%d, k=%d\n", i, j, k); }
};
```

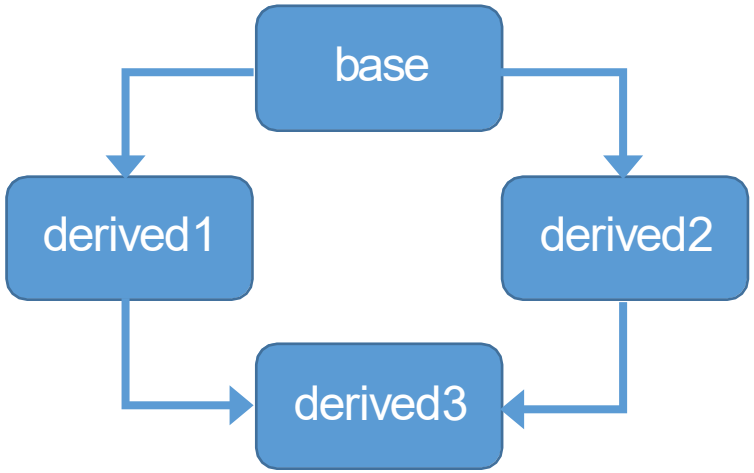
```
derived::derived(int x, int y, int z): base1(y), base2(z)
{
   k=x;
   std::cout << "Constructing derived" << std::endl;
}
```

```
int main ()
{
   derived obj(3, 4, 5);
   obj.show();
   return 0;
}
```

```
i=4, j=5, k=3
```

Multiple inheritance: the "diamond problem"

```
class base
{
 public:
   int i;
};
class derived1: public base
{
 public:
   int j;
};
class derived2: public base
{
 public:
   int k;
};
class derived3: public derived1, public derived2
{
 public:
   int sum;
};
```




```
int main ()
{
   derived3 obj;

   obj.j = 10;
   obj.k = 10;
   obj.i = 10;

   return 0;
}
```

from derived1

from derived2

which i?

Multiple inheritance: the “diamond problem”

Two solutions to the ambiguity created by the diamond problem:

1. Apply the scope resolution operator, or
2. Prevent two copies of `base` being included in `derived3`

The “diamond problem”: solution 1

University of Exeter

```
class base
{
 public:
   int i;
};
class derived1: public base
{
 public:
   int j;
};
class derived2: public base
{
 public:
   int k;
};
class derived3: public derived1, public derived2
{
 public:
   int sum;
};
```

derived3 contains pointers to two copies of base

Apply the scope Resolution operator

```
int main ()
{
   derived3 obj;

   obj.j = 1;
   obj.k = 2;
   obj.derived1::i = 10;

   return 0;
}
```

Diamond problem: virtual base class

Prevent two copies of base being included in derived3

```
class base
{
 public:
   int i;
};
class derived1: virtual public base
{
 public:
   int j;
};
class derived2: virtual public base
{
 public:
   int k;
};
class derived3: public derived1, public derived2
{
 public:
   int sum;
};
```

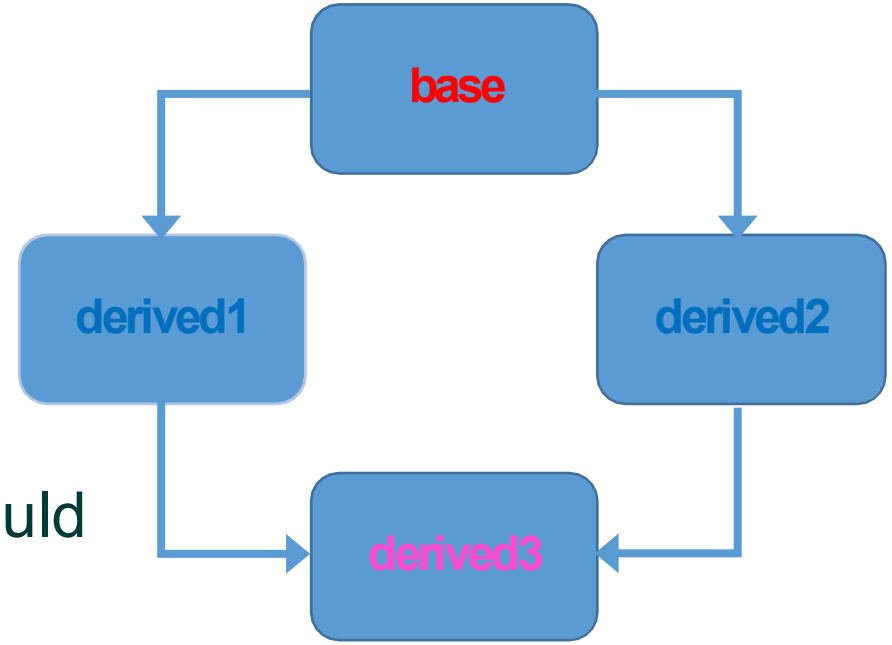
```
int main ()
{
   derived3 obj;

   obj.j = 1;
   obj.k = 2;
   obj.i = 10;

   return 0;
}
```

Virtual base classes

- If derived1 and derived2 inherit virtually from base, then objects of derived3 contain only **one set** of member variables from base

- The virtual base acts as if it were the direct base class of derived3

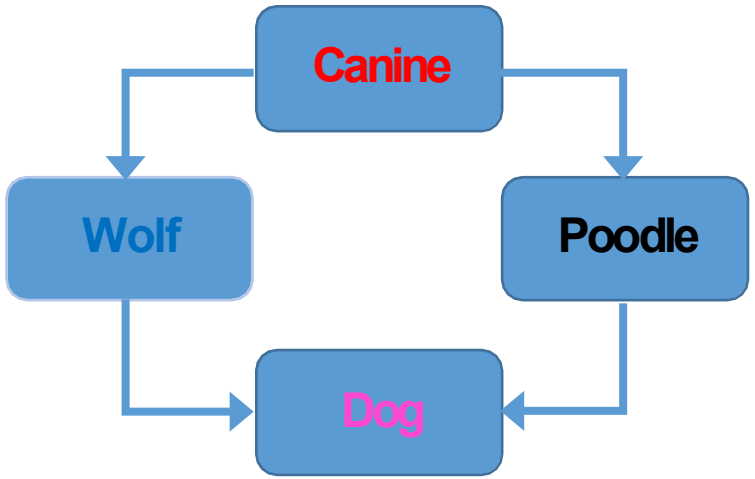
- Base classes common through the hierarchy should be declared as virtual



Virtual member functions

Virtual member function allows derived classes to override its implementation.

```
class Canine
{
protected:
    std::string name;
 public:
    Canine(std::string n) { name = std::string(n); }
    virtual void call() { std::cout << name << std::endl; }
};

class Wolf: virtual public Canine
{
 public:
    Wolf(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Howl " << name << std::endl; }
};
class Poodle: virtual public Canine
{
 public:
    Poodle(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Woof " << name << std::endl; }
};
class Dog: public Wolf, public Poodle
{
 public:
    Dog(std::string n): Canine(n), Wolf(n), Poodle(n) {}
    void call() { std::cout << "Bark " << name << std::endl;}
};
```




```
int main ()
{
    Dog pet("Fido");
    pet.call();
   return 0;
}
```

"Bark Fido"

Virtual member functions

University of Exeter

```
class Canine
{
protected:
    std::string name;
 public:
    Canine(std::string n) { name = std::string(n); }
    virtual void call() = 0;
};
class Wolf: virtual public Canine
{
 public:
    Wolf(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Howl " << name << std::endl; }
};
class Poodle: virtual public Canine
{
 public:
    Poodle(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Woof " << name  << std::endl; }
};
class Dog: public Wolf, public Poodle
{
 public:
    Dog(std::string n): Canine(n), Wolf(n), Poodle(n) {}
    void call() { std::cout << "Bark " << name << std::endl;}
};
```

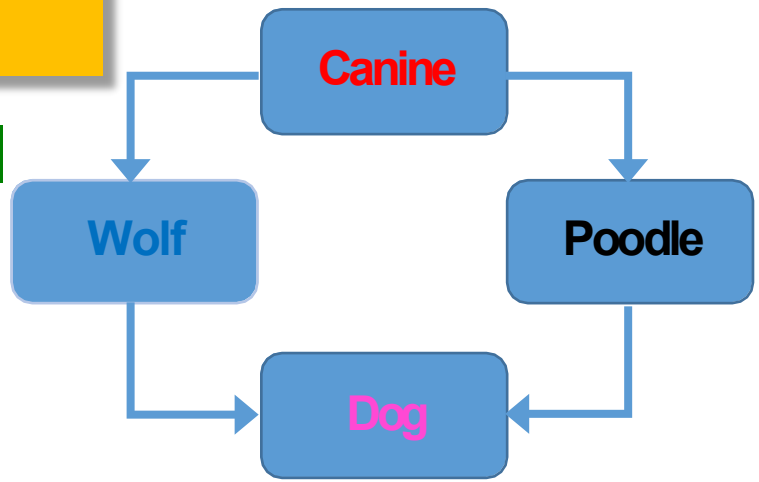
Ensure call() is overridden in at least one derived class

Virtual member functions

```
class Canine
{
protected:
    std::string name;
 public:
    Canine(std::string n) { name = std::string(n); }
    virtual void call() { std::cout << name << std::endl; }
};

class Wolf: virtual public Canine
{
 public:
    Wolf(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Howl " << name << std::endl; }
};
class Poodle: virtual public Canine
{
 public:
    Poodle(std::string n): Canine(n) {}
    virtual void call() { std::cout << "Woof " << name << std::endl; }
};
class Dog: public Wolf, public Poodle
{
 public:
    Dog(std::string n): Canine(n), Wolf(n), Poodle(n) {}
    void call() { std::cout << "Bark " << name << std::endl;}
};
```

```
int main ()
{
    Dog pet("Fido");
    Canine *canis = &pet;
    canis->call();
    return 0;
}
```

"Bark Fido"



Compiler may not be able to tell which class will implement the member function (only declarations may be available).

So member function only bound at runtime.

Summary

- Access specifiers (public, protected, private) control how base members are exposed in derived classes.
- Base class constructors run **before** derived constructors; destructors run in the **reverse order**.
- Constructor arguments for base classes must be passed using the **member initializer list**.
- **Multiple inheritance** allows a class to inherit from more than one base class.
- **Virtual base class** ensure only one shared base instance exists.
- **Virtual functions** enable runtime polymorphism - function calls are resolved based on the actual object type.